DOCUMENT STUDIO

Complete Product, Architecture, UI/UX and Codex Build Blueprint

Local-first document and PDF workspace

Re-audited master specification

132 planned capabilities in one complete edition

Windows desktop first - optional web and mobile later

Black-and-white print edition

Prepared for V. Sai Rohith

Version 2.0.1 - 22 July 2026

Re-audit outcome

The earlier work was a strong starting point, but it was not ready to begin implementation without correction. The previous report still used the former name, several external-app updates were not verifiably complete, and the architecture/UI/model/dependency handoffs lacked enough operational detail. This master blueprint corrects those gaps and makes Document Studio the sole canonical name.

Area	Final decision
Product	Document Studio - one complete personal edition
First platform	Windows desktop; macOS follows
Desktop stack	Tauri 2.11.x + React 19.2.7+ + TypeScript + Vite 8.1.5+ + Rust
Processing	Local by default; optional browser/cloud/external paths are explicit
Viewer	PDF.js with progressive rendering and virtualized thumbnails
Core engines	qpdf, libvips, OCRmyPDF/Tesseract, LibreOffice
AI	Optional, provider-neutral, local-first, page-cited and consent-gated
Design	Precision Paper - calm, professional, accessible and document-first
Implementation	Codex vertical slices; Phase 0 before feature expansion

Planning package status: complete enough to create the private repository and start Phase 0. Live publication to Notion, Figma, GitHub or Canvs must be confirmed by their connectors; the import-ready assets are included in this package.

Contents

1. Audit and Completion Record

Audit date: 22 July 2026 Canonical name: Document Studio Package version: 2.0.1-final-recheck

Finding

The previous work was a useful foundation but was not complete enough to begin implementation safely. The older specification still used the former product name, the live Notion/whiteboard state was not verified, the UI/UX handoff was incomplete, the Hugging Face and GitHub candidates were not governed by an explicit adoption policy, and the architecture did not clearly separate the desktop runtime from a future cloud backend.

Corrections completed in this package

- Renamed the product, repository, executable, workflow extension and all current documents to Document Studio.
- Preserved the old reports only under attachments/archive/ as historical inputs.
- Defined the platform sequence and a single canonical desktop architecture.
- Added a 132-item unified feature catalogue with phase, engine and priority.
- Added screen specifications, user journeys, component inventory, design tokens, accessibility criteria and a Figma build recipe.
- Added system, execution, data, security, performance and deployment architecture.
- Added SQLite/PostgreSQL schemas and API/operation contracts.
- Added a dependency decision register and local model registry.
- Added Codex prompts, AGENTS instructions, validation scripts and a Tauri starter scaffold.
- Added a browser-openable UI prototype and architecture/flow diagrams.
- Added Notion import folders containing the blueprint, source research, old reports, current report, diagrams and whiteboard assets.

What is deliberately not claimed

- No live GitHub repository was created from this environment.
- No live Figma file was edited from this environment.
- No Hugging Face model was downloaded or benchmarked; candidates are documented with gates.
- No external Notion or Canvs board update is claimed unless the connector confirms it.
- The full production application is not implemented; this package is the audited plan and starter scaffold.

Gate before implementation

Phase 0 can begin only after the project directory is opened in Codex, the starter is committed to a private repository, and the toolchain/dependency checks in docs/22-IMPLEMENTATION-READINESS-CHECKLIST.md are completed.

2. Product Charter

Product statement

Document Studio is a fast, beautiful and private document workspace for converting, organizing, optimizing, editing, OCR, forms, signatures, security, automation and optional document intelligence. It should feel like one coherent application rather than dozens of disconnected upload pages.

Primary outcome

A user can drop one or many files, choose an operation, inspect the exact pages/settings, run a cancellable job, verify the result and save a new file without losing the original.

Core principles

1. Local first. Routine PDF and image operations require no upload.
2. Truthful completion. Success appears only after output verification.
3. Speed by architecture. Avoid upload latency, warm heavy engines, stream progress, cache previews and choose the fastest safe execution path.
4. One complete edition. No artificial feature partitions in the personal build.
5. Preview before irreversible work. Redaction, signatures, destructive page changes and security settings require an explicit review.
6. Original product design. Match useful capabilities and performance expectations, not another product's brand, copy, assets or source code.
7. Progressive power. Simple defaults first; advanced controls are available without overwhelming routine users.
8. Accessible by default. Keyboard, screen reader, high contrast, reduced motion and zoom are design constraints.

Platform decision

Order	Platform	Why
1	Windows desktop	Fastest local processing, simplest privacy story, highest value for school/office workflows
2	macOS desktop	Reuse the Tauri/React workbench with platform packaging and Apple Silicon model options
3	Optional web app	Reach and shareability; browser-local fast path plus cloud workers for unsupported/heavy tasks
4	Android/iOS companion	Scan, capture, annotate, sign, send to desktop/web; do not duplicate every heavy converter initially

Success measures

- First meaningful result in under 60 seconds for a new user.
- Routine merge/split/rotate jobs complete locally without network access.
- No false-success reports in automated recovery tests.
- 95% of core workflows operable by keyboard.
- Performance budgets in 08-PERFORMANCE-ARCHITECTURE.md met on the reference machine.

Figure 1. Product delivery sequence from foundation to optional ecosystem.

3. Personas and Jobs to Be Done

Primary persona: Rohith / school administrator

Needs to combine circulars, compress scanned records, convert Office files, add page numbers/watermarks, OCR English/Telugu/Hindi documents, protect sensitive files, sign documents and repeat routine workflows without uploading private school records.

Secondary personas

- Teacher: prepare, merge, annotate and print learning material.
- Office administrator: convert, batch rename, protect, redact and archive documents.
- Student: scan, compress, convert and organize assignments.
- Power user/developer: run repeatable jobs through workflows or CLI.

Highest-value jobs

1. Combine many documents in the right order without damaging originals.
2. Make large PDFs small enough to share while preserving readability.
3. Turn scans into searchable PDFs in English, Hindi and Telugu.
4. Convert Word/Excel/PowerPoint documents to consistent PDFs.
5. Add page numbers, watermarks, signatures and protection in a single reviewed flow.
6. Remove sensitive information so it cannot be recovered.
7. Re-run recurring document recipes without reconfiguring every step.
8. Ask a document a question and see the exact supporting pages.

Anxiety-reducing UX requirements

- Always show where the result will be saved.
- Always show whether processing is local or external.
- Never hide destructive implications behind vague labels.
- Provide plain-language recovery steps when a file is encrypted, damaged or unsupported.
- Keep a visible cancel action for long-running jobs.

4. Unified Feature Catalogue

The canonical machine-readable catalogue is feature-catalog.csv. It currently contains 132 planned capabilities. Phase numbers are implementation order, not access tiers.

Phase summary

- Phase 0: application foundation, privacy, accessibility, diagnostics, job lifecycle and history.
- Phase 1: structural PDF operations and basic image conversion.
- Phase 2: compression, Office conversion, overlays, batch processing and protection.
- Phase 3: editing workspace, forms, signatures and OCR.
- Phase 4: true redaction, sanitization, structured extraction and comparison foundations.
- Phase 5+: archival, digital signatures, advanced conversion, workflows, AI, web/mobile and plugin platform.

Product rule

Every implemented feature is available in one complete personal edition. “Must/Should/Could” expresses implementation priority only.

5. Platform Strategy

Decision

Build the desktop product first. It gives the shortest path to speed and privacy because files stay on the machine and mature native command-line engines can be used directly. Build a web version only after the operation contract and verification model are stable.

Shared product layers

- Shared React component library and design tokens.
- Shared operation manifests and JSON Schemas.
- Shared naming, validation, warning and error language.
- Shared test fixtures and acceptance cases.

Platform-specific execution

Desktop

- Tauri 2 shell and Rust orchestration.
- Direct adapters to signed/bundled qpdf, libvips and selected utilities.
- Optional separately installed LibreOffice, OCRmyPDF and language packs.
- SQLite history/presets/workflows.

Web

- Browser-local path for operations that can be safely implemented with PDF.js/pdf-lib/WASM and fit browser memory.
- Cloud worker path for Office conversion, heavy OCR, AI and very large files.
- Clear local/cloud badge before execution.

Mobile

- Capture, crop, dewarp, rotate, annotate, sign, simple organize/compress.
- Share-sheet integration and optional handoff to desktop/web.
- Heavy Office/OCR/AI features can be deferred or delegated with consent.

Why not build every platform at once

Simultaneous desktop, web and mobile builds multiply packaging, testing, security and performance variables before the core operation lifecycle is proven. One strong vertical slice is more valuable than three incomplete shells.

6. UI/UX Strategy - Precision Paper

Visual concept

Document Studio should look like a modern professional workspace: crisp geometry, strong typography, paper-like content surfaces, quiet chrome and one confident accent. It must be more polished than a grid of utility cards without becoming decorative or slow.

Experience principles

- Drop once, work continuously. A document remains open while the user moves between compatible operations.
- Contextual tools. Show controls relevant to the selected files/pages, not the entire catalogue.
- Progressive disclosure. Default presets are understandable; advanced controls expand on demand.
- Confidence through preview. Before/after, page thumbnails, output summary and verification results are visible.
- Locality is visible. A compact badge says “On this device” or names the external provider.
- One primary action. Each screen has one clear execution button.
- Recoverable mistakes. Reordering/removal/crop/overlay changes remain undoable until export.

Brand direction

- Name: Document Studio.
- Descriptor: “Your private document workspace.”
- Visual metaphor: stacked pages forming a precise aperture/letter D.
- Tone: direct, calm, respectful and technically honest.
- Avoid: gradients for decoration, glassmorphism, excessive shadows, cartoon illustrations, red as a brand color, and marketing claims without measurement.

Layout

- Collapsible left navigation rail.
- Context panel for files/pages.
- Central document canvas.
- Right inspector for operation settings and verification.
- Persistent bottom job tray.
- Global command palette and search.

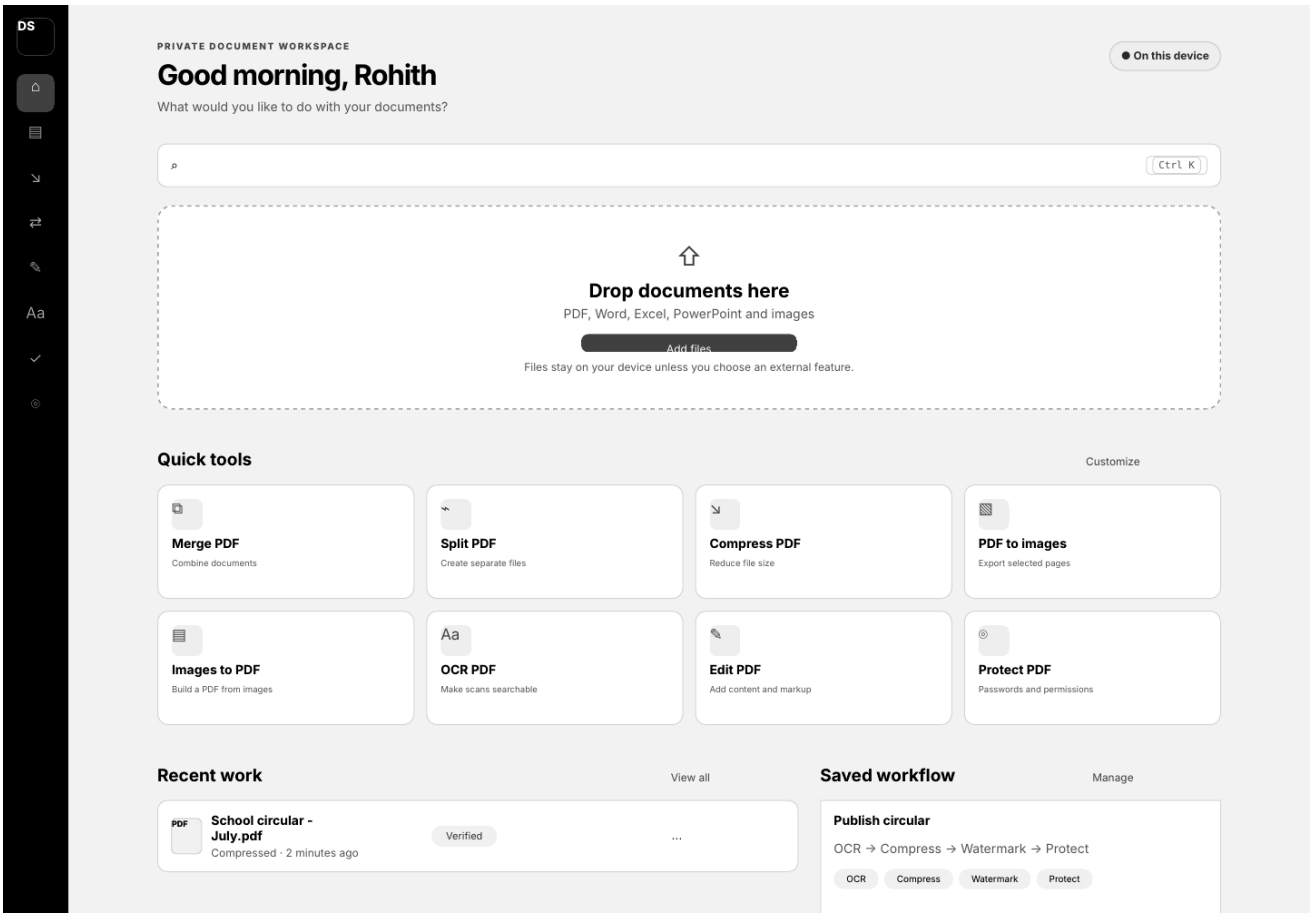

Figure 2. Home-screen design direction: local status, drop zone, quick tools, recent work and saved workflows.

7. Information Architecture

Main navigation

1. Home
2. Organize
3. Optimize
4. Convert
5. Edit
6. OCR
7. Forms & Sign
8. Protect
9. Automate
10. AI Lab (disabled until enabled)
11. History
12. Settings

Home

- New drop zone.
- Search/command bar.
- Pinned quick tools.
- Recent work.
- Saved workflows.
- System status: local engines, storage, offline/privacy mode.

Workbench anatomy

- Top bar: document name, save destination, undo/redo, help, locality badge.
- Left panel: input files and page thumbnails; multi-select, drag reorder, page warnings.
- Center canvas: PDF/image preview with selection and editing overlays.
- Right inspector: operation-specific settings, presets, output naming, validation messages.
- Bottom job tray: queued/running/completed jobs, stage, progress, cancel, retry, open/reveal.

Navigation rule

Catalog categories help discovery, but once files are open the workbench should recommend compatible actions without forcing the user back to Home.

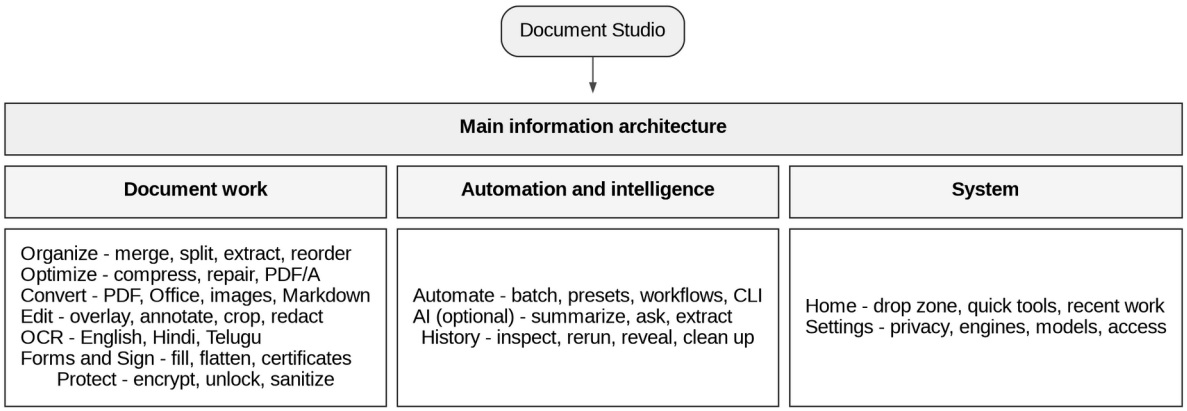

Figure 3. Main information architecture.

8. Screen Specifications

Home

- Greeting and compact privacy status.
- Universal “Search tools and commands” field.
- 640-760 px wide drop zone with local/cloud explanation.
- Eight configurable quick-tool cards.
- Recent jobs table with status, operation, time, output and action menu.
- Saved workflow cards with step chips and run button.

Tool setup screen

- Files list with validation state.
- Operation explanation in one sentence.
- Smart defaults selected.
- Advanced section collapsed.
- Output path and naming template always visible.
- “Run” button states exact action, e.g., “Merge 8 pages.”

Workbench

- Virtualized thumbnails for large files.
- First page appears while remaining previews render progressively.
- Canvas supports fit page/width, zoom, search and keyboard page navigation.
- Inspector uses sections: Selection, Operation, Output, Verification.
- Dangerous choices use warning copy, not only color.

Result screen

- Verified status and checks performed.
- Output path, size, page count and processing time.
- Open, reveal, compare, run another and add-to-workflow actions.
- Warnings remain visible; success is not shown when verification failed.

Settings

- General, Appearance, Privacy, Storage, Dependencies, OCR languages, AI providers, Updates, Accessibility, Diagnostics.
- Global network disable switch.
- Clear history/temporary storage with scope preview.

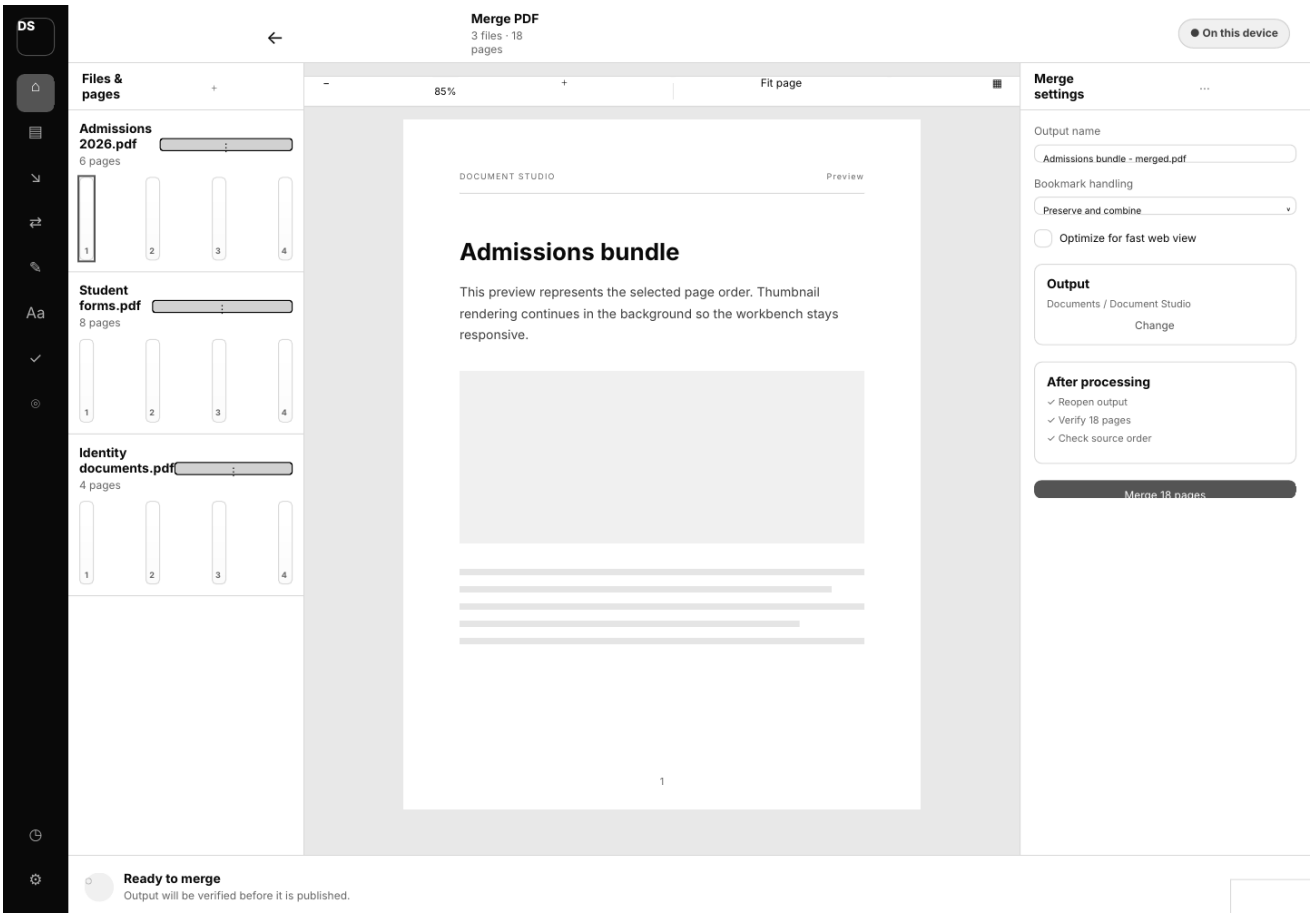

Figure 4. Merge workbench concept with files/pages, document canvas, inspector and persistent job tray.

9. Performance Architecture

Performance objective

The product should feel immediate for common local tasks and explain unavoidable delays for OCR, Office conversion and AI. Performance is measured on a named reference machine and with published test documents.

Reference budgets (initial targets)

Interaction	Target
Warm desktop launch to usable Home	<= 1.5 s
Cold desktop launch	<= 3.0 s
First visible page of a normal PDF	<= 1.0 s after selection
100-page thumbnail strip usable	<= 3.0 s with progressive rendering
Merge ten 10-page text PDFs locally	<= 2.0 s excluding antivirus overhead
Rotate/reorder metadata operation	<= 1.5 s
Cancel acknowledgement	<= 250 ms; worker termination/cleanup reported separately
UI input response while jobs run	<= 100 ms
Job progress update	4-10 Hz, throttled to avoid UI overhead

These are engineering targets, not guarantees. They must be calibrated on Windows hardware used by the owner.

Techniques

- Keep structural operations in-process or in a warm trusted sidecar/CLI adapter.
- Avoid copying large inputs unless the operation or safety model requires it.
- Memory-map/read streams where supported; never load a multi-gigabyte file into the UI process.
- Render first page and visible thumbnails first; virtualize the rest.
- Cache thumbnails by file fingerprint and renderer version.
- Separate CPU, I/O and memory-heavy queues with concurrency limits.
- Keep LibreOffice and OCR warm only when resource budgets allow.
- Use libvips for low-memory streaming image transforms.
- Publish outputs with atomic rename after verification.
- Benchmark every operation with 10, 100, 500 and 1,000-page fixtures where practical.

Fast-path router

1. Inspect file, size, encryption, page count and operation.
2. Choose browser-local, desktop-local or cloud worker according to capability and policy.
3. Estimate work units and resource risk.
4. Run with bounded concurrency.
5. Verify output and record timing by stage.
6. Feed benchmarks into regression thresholds.

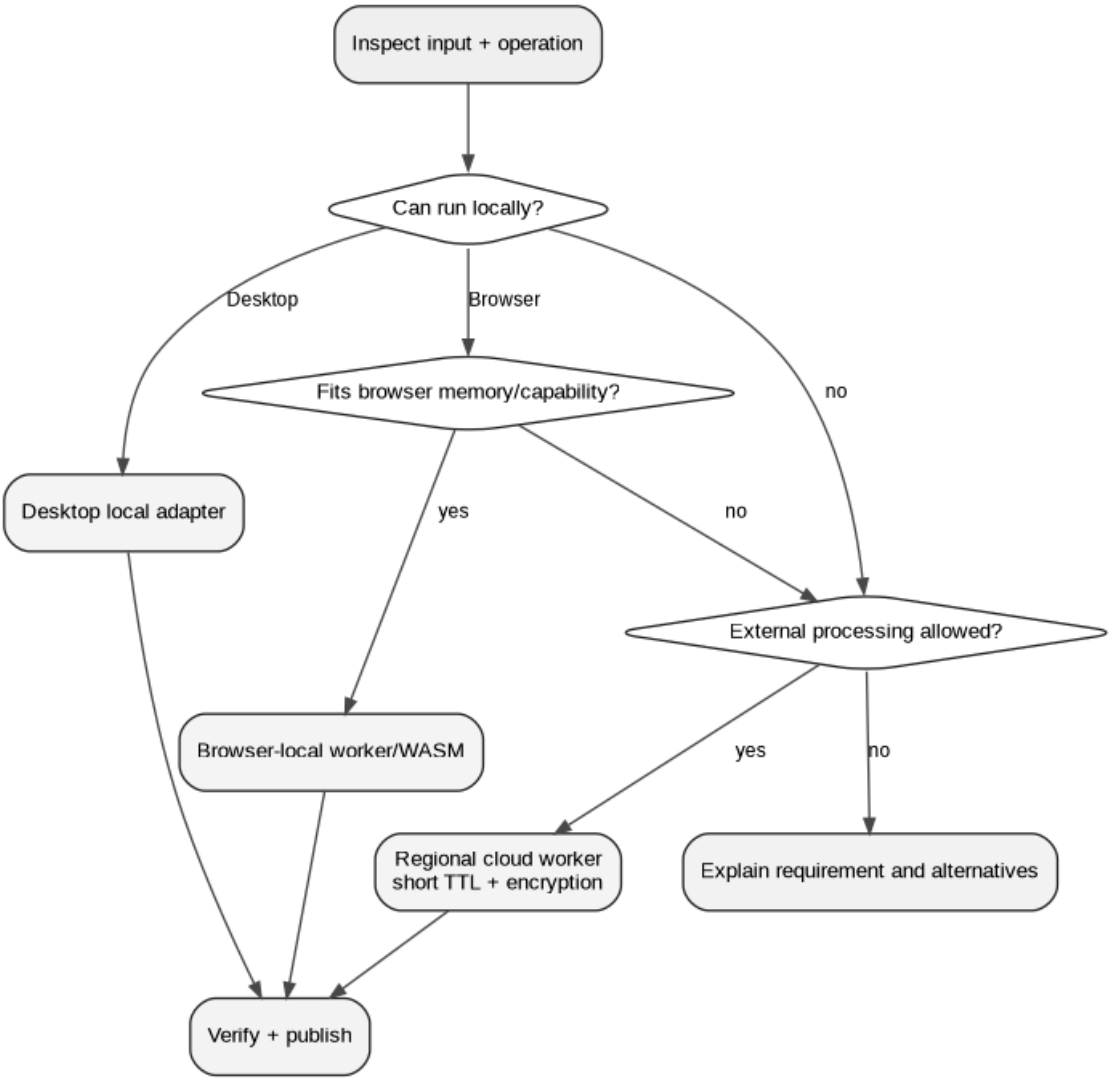

Figure 5. Execution-path router for desktop, browser-local and optional cloud work.

10. System Architecture

Desktop architecture

The desktop app uses Tauri as the trusted boundary. React renders the workbench. The UI invokes allow-listed typed commands. Rust validates paths/settings, creates a private job workspace, selects an operation adapter, streams structured progress, supports cancellation, verifies outputs and records history.

Components

- React workbench: presentation, local state, validation display, thumbnails and editor scene graph.
- Tauri command layer: secure IPC and capabilities.
- Rust job engine: state machine, queue, resource scheduler, cancellation, recovery, audit.
- Operation registry: manifests, schemas, dependencies and adapter factories.
- Adapters: qpdf, libvips, LibreOffice, OCRmyPDF/Tesseract, optional Docling/model services.
- SQLite repository: settings, jobs, presets, workflows, dependencies and model state.
- Workspace manager: per-job directories, staging, atomic publication and cleanup.

Future web architecture

- Static React web app/CDN.
- API gateway and Go control plane.
- PostgreSQL metadata, Redis/Valkey queue coordination and object storage.
- Isolated worker pools by engine/risk profile.
- Regional routing and explicit retention policy.
- Browser-local execution where safe and fast.

Architectural boundary

The local desktop should not depend on the future Go service. Both implement the same operation contract and schemas, but desktop remains fully useful offline.

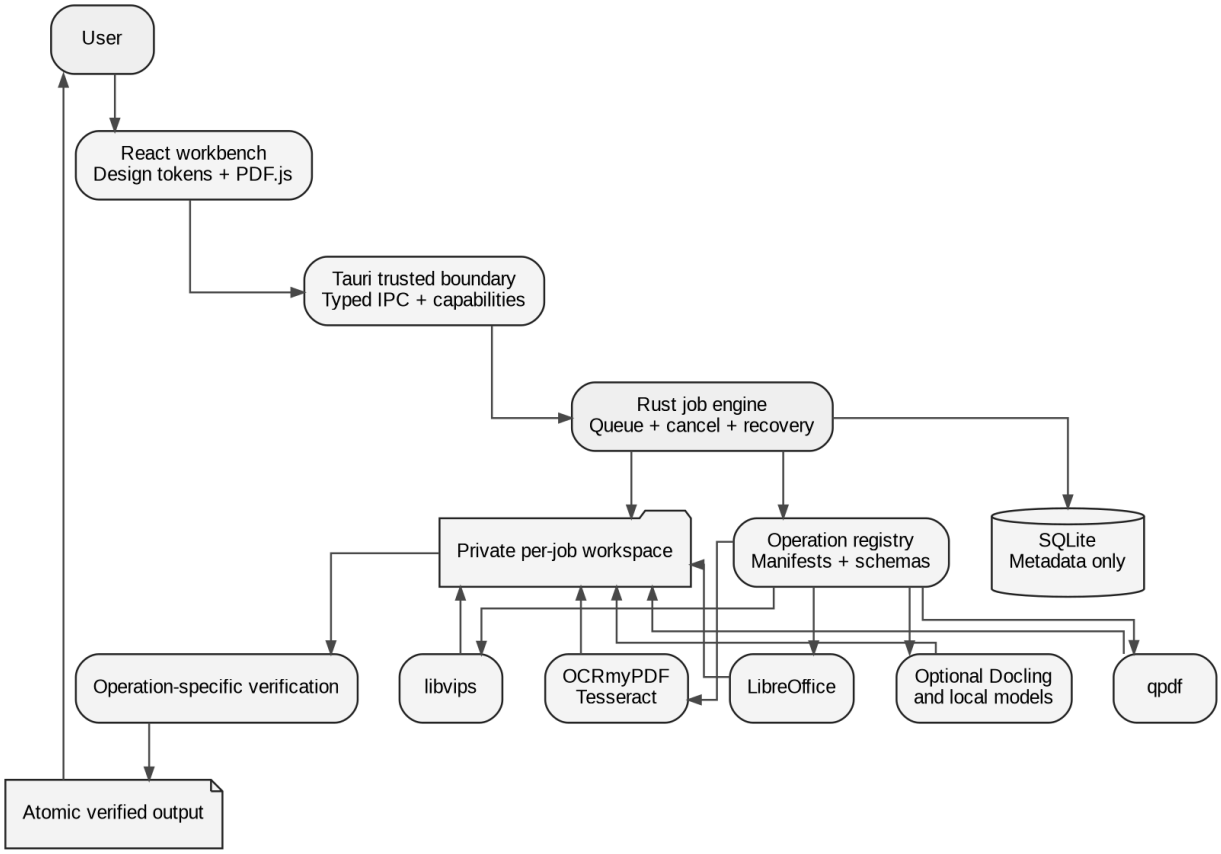

Figure 6. Canonical desktop system architecture.

11. Unified Operation Contract

Every tool must implement the same lifecycle.

Manifest fields

- id and semantic version.
- User-facing name, category and description.
- Accepted input types, multiplicity, page-selection rules and maximum tested sizes.
- Typed settings schema and defaults.
- Dependency and platform capabilities.
- Risk level and whether preview/confirmation is required.
- Output type and verification rules.

Lifecycle

1. inspect - collect safe metadata without mutating input.
2. preflight - validate files, passwords, dependencies, storage and settings.
3. estimate - work units, likely duration class and output-size range when possible.
4. plan - resolve adapter, arguments and workspace paths.
5. execute - cancellable processing with structured progress events.
6. verify - reopen output and assert operation-specific invariants.
7. publish - atomically move verified output to the destination.
8. audit - store timings, versions, fingerprints, warnings and result paths.
9. cleanup - remove temporary data on every terminal path.

Standard progress event

```
{
 "jobId": "uuid",
 "operationId": "pdf.merge",
 "state": "running",
 "stage": "assembling-pages",
 "completedUnits": 32,
 "totalUnits": 80,
 "message": "Adding page 32 of 80",
 "cancellable": true
}
```

Verification examples

- Merge: output opens, expected total pages, expected source order and no unintended encryption.
- OCR: output opens, page count preserved, text layer exists on representative/all pages as configured.
- Redaction: target regions are removed from text extraction and pixels are sanitized according to mode.
- Protect: wrong password fails, correct password opens, requested permission flags match.

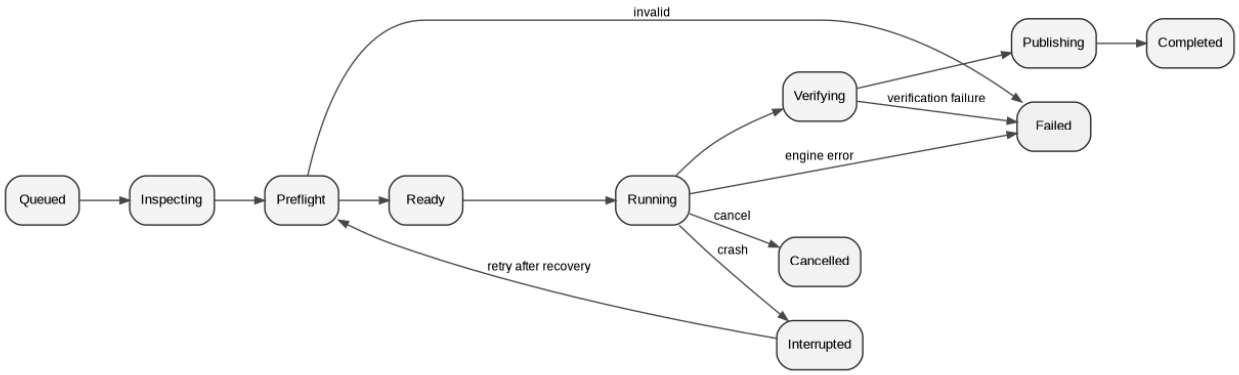

Figure 7. Verified job lifecycle and alternate terminal states.

12. Data and Database Design

Desktop storage

SQLite stores metadata only. Documents remain in user-selected locations or short-lived job workspaces.

Core entities

- jobs: state, operation, timestamps, progress, stage, destination, verification and error.
- job_inputs: path, fingerprint, MIME, size, page count and password reference.
- job_outputs: staging/final path, MIME, size, page count and verification result.
- presets: operation-scoped settings JSON.
- workflows: versioned directed sequence of operation steps.
- workflow_runs: resolved variables and per-step job IDs.
- dependencies: path, version, health and capabilities.
- models: model ID, revision, license, files, verification and install state.
- settings: scoped key/value store with migration version.
- signature_identities: references to protected local assets/certificates; no plaintext secrets.

Retention

- Job history is configurable.
- Temporary workspaces are removed after terminal states and reconciled at startup.
- Extracted text/embeddings are opt-in and scoped to AI features.
- Passwords/API keys/certificate secrets are stored in the OS credential store, referenced by opaque IDs.

Future cloud storage

PostgreSQL stores account, job and policy metadata. Object storage holds encrypted inputs/outputs with short TTLs. Redis/Valkey coordinates queues and locks. Storage region and deletion deadline are recorded per object.

13. API and IPC Design

Desktop IPC

The React app may call only named Tauri commands with serializable typed payloads. It cannot execute arbitrary binaries or write unrestricted paths.

Command groups

- files.inspect
- operations.list
- jobs.create
- jobs.cancel
- jobs.get
- jobs.subscribe
- history.list
- history.delete
- dependencies.scan
- settings.get/set
- models.list/install/remove

Future cloud API

- REST/JSON for job submission, metadata and downloads.
- Server-sent events or WebSocket for progress.
- Idempotency key for submissions.
- Pre-signed upload/download URLs with short expiry.
- Explicit operation/version and schema version.
- Authentication, rate limits and regional policy.

Error envelope

```
{
 "code": "INPUT_ENCRYPTED",
 "title": "This PDF needs a password",
 "detail": "Provide the opening password to continue.",
 "inputIndex": 1,
 "stage": "preflight",
 "retryable": true,
 "helpId": "pdf-passwords"
}
```

14. Security, Privacy and Threat Model

Trust boundaries

- Untrusted documents and archives.
- UI/webview process.
- Tauri/Rust trusted core.
- Third-party document engines.
- Optional external AI/cloud providers.
- User filesystem and OS credential store.

Required controls

- Allow-list executables and operation arguments.
- Pass arguments as arrays; never concatenate shell strings.
- Canonicalize paths and enforce destination/workspace roots.
- Reject path traversal, symlink escape and dangerous archive entries.
- Isolate each job; use least privilege and resource limits where supported.
- Validate magic bytes and parser output, not only extensions.
- Patch bundled engines and record exact versions/hashes.
- Treat PDFs, images, fonts, Office files and archives as attacker-controlled.
- Disable network by default for document processing.
- Require per-job consent for external processing and show provider/data scope.
- Never log passwords, keys, document text or raw model prompts by default.
- Verify signed updates and bundled sidecar checksums.

Redaction safety

Visual cover-up is not redaction. The operation must remove underlying text/objects or rasterize/sanitize the affected area, then test extraction and visual output. The UI requires a final irreversible confirmation.

Privacy modes

- Normal local mode.
- Strict offline mode (all network disabled, including update/model checks).
- External-provider mode enabled per feature/provider with explicit scope.

15. Dependency and License Register

The dependency register distinguishes adopt, evaluate, optional service and inspiration only. Versions are verified at implementation time and pinned with checksums.

Component	Use	Status	License/notes
Tauri 2.11.x	Desktop shell and secure IPC	Adopt	MIT/Apache-2.0; CLI 2.11.4 and JS API 2.11.1 were current at the 22 July 2026 recheck
React 19.2.7+	UI	Adopt	MIT; pin a patched 19.2.x or later
Vite 8.1.5+	Front-end build	Adopt	MIT; Vite 8 uses Rolldown; pair with @vitejs/plugin-react 6.x
PDF.js 5.7.284+	Viewer, text layer, annotations	Adopt	Apache-2.0
qpdf 12.3.2+	Structural PDF transforms/encryption	Adopt	Apache-2.0; verify release signatures/checksums
libvips 8.18.2+	Image conversion/compression	Adopt	LGPL-2.1-or-later; dynamic-linking/distribution review
OCRmyPDF 17.8.x	OCR orchestration	Evaluate as managed optional worker	MPL-2.0; native Windows needs Python, Tesseract and Ghostscript; packaging and sandboxing gate required
Tesseract 5.5.2+	OCR engine/language packs	Adopt	Apache-2.0; Windows installer is third-party, so binary provenance must be governed
LibreOffice 26.2.4+	Office conversion	Adopt external dependency	MPL-2.0 and bundled notices; detect installed version and isolate profile directories
Gotenberg 8.32+	Server-side Office/HTML conversion	Optional web service	MIT; container deployment; keep patched because conversion surfaces process untrusted files
pdf-lib	Browser overlays/simple creation	Evaluate/adopt for narrow cases	MIT; not full arbitrary editing/decryption
pdfcpu 0.12+	Go-side PDF operations/signature evaluation	Evaluate	Apache-2.0; project describes itself as evolving
veraPDF	PDF/A validation	Evaluate	dual-license/distribution review required
Docling	Structured document parsing	Evaluate	MIT; model licenses separate
Granite Docling 258M	Local document structure extraction	Evaluate	Apache-2.0; English model
multilingual-e5-small	Multilingual embeddings	Evaluate	MIT; 512-token chunk limit
UI UX Pro Max	Design guidance for Codex	Adopt as development skill	MIT; install through uipro-cli
Tokens Studio	Figma design-token sync	Recommend	MIT
Iconify	Icon exploration/import	Recommend with icon-set license review	Plugin/repository terms plus individual set licenses
Design Lint	Figma consistency checks	Recommend	MIT
Stirling-PDF	Feature inspiration/benchmarking	Inspiration only	Do not vendor or copy; separate open-core/security considerations

Adoption gate

A dependency is not production-approved until licensing, update channel, binary provenance, sandboxing, performance, failure behavior and output verification are documented in an ADR.

Version-policy note

The versions above are recheck baselines, not permanent pins. Before each release, update the dependency lock, verify upstream signatures/checksums and advisories, run the compatibility matrix, and record the adopted revision in an ADR and software bill of materials.

16. Hugging Face and Local Model Plan

Models are optional accelerators, not the foundation of ordinary PDF tools.

Approved for evaluation

IBM Granite Docling 258M

- Purpose: document layout, tables, formulas and structured conversion.
- License: Apache-2.0.
- Size: approximately 530 MB model repository.
- Caveat: model card identifies English as the language; do not assume Telugu/Hindi quality.
- Gate: benchmark against rule-based/Docling pipeline on owned test documents before enabling.

intfloat/multilingual-e5-small

- Purpose: local semantic search/retrieval for Ask Document.
- License: MIT.
- Approximately 117M parameters, 384-dimensional embeddings and 512-token input limit.
- Supports a broad multilingual training setup, but Telugu/Hindi retrieval must be benchmarked.

Tesseract language data

- eng, hin, tel are the initial OCR packs.
- Benchmark accuracy, speed and installation size on representative school documents.

PaddleOCR Telugu recognition model

- Benchmark-only alternative for Telugu recognition.
- Do not ship until integration, accuracy, license and runtime footprint beat or complement Tesseract.

Model governance

- Pin repository ID and immutable revision.
- Record model card, license, size and checksums.
- Download only after user action; support remove/update.
- Store under a dedicated model cache, not inside job folders.
- Run local models in a constrained worker process.
- Display model/provider and whether text/images leave the device.
- Evaluate accuracy by language and document type; never advertise universal accuracy.

17. Test and Quality Strategy

Layers

- Unit tests: schemas, range parsing, naming, path validation, job transitions, settings migrations.
- Adapter integration: known input -> operation -> verified output.
- Golden visual tests: render selected outputs and compare with tolerance.
- Cross-renderer tests: PDF.js plus at least one independent desktop/PDFium renderer.
- Security tests: malformed files, traversal, command injection, archive bombs, cancellation and secret logging.
- Recovery tests: kill the process during every stage and relaunch.
- Performance tests: named fixtures and thresholds by stage.
- Accessibility tests: keyboard, focus, screen-reader names, 200% zoom, high contrast and reduced motion.
- Packaging tests: clean Windows/macOS VM, upgrades, missing dependencies and uninstall.

Definition of done for a feature

1. Manifest and typed settings complete.
2. UI explains accepted inputs, locality and risky choices.
3. Preflight catches predictable failures.
4. Progress and cancellation work.
5. Output is staged and verified.
6. Original remains unchanged by default.
7. Temporary files are cleaned in all terminal paths.
8. Success and at least three meaningful failure tests exist.
9. History is useful and does not leak content/secrets.
10. User documentation and performance evidence are updated.

18. Delivery, CI/CD and Observability

Branch and release flow

- Protected main.
- Short-lived feature branches.
- Pull request requires specification validation, formatting, typecheck, tests and security/dependency review for new engines.
- Signed tags and release artifacts.
- Reproducible sidecar acquisition with checksums.

Desktop packaging

- Windows MSI/NSIS first.
- macOS universal/architecture-specific package later.
- Optional engine bundles separated when licensing or size requires it.
- Application update channel is signed and can be disabled.

Local observability

- Structured logs with correlation/job ID.
- Stage timings and engine version.
- No document text, password, key or prompt body by default.
- User-controlled diagnostics export with redaction.
- Local benchmark history to detect regressions.

Future cloud observability

- Metrics: queue wait, processing time by stage, failure code, worker saturation, upload/download duration and cleanup success.
- Distributed tracing without document content.
- Security audit logs and deletion proofs.
- SLOs defined only after real load tests.

19. Roadmap and Milestones

Phase 0 - Foundation

Tauri shell, React design system, PDF viewer placeholder, job lifecycle, SQLite, secure workspace, diagnostics, recovery, prototype parity and diagnostic.copy reference operation.

Exit: a cancellable job is created, processed, verified, published, recorded and recovered correctly after forced termination.

Phase 1 - Core PDF

Merge, split, extract, remove, reorder, rotate, linearize, PDF-to-images, images-to-PDF and naming templates.

Exit: golden and failure tests pass; performance budgets are measured.

Phase 2 - Optimize, convert and protect

Compression, Office-to-PDF, text/Markdown/HTML-to-PDF, page numbers, watermark, metadata, encrypt/unlock, batch runner.

Phase 3 - Workbench, OCR, forms and signatures

Viewer/editor interaction, overlays, crop, annotations, fill/flatten forms, signature placement, OCR in English/Hindi/Telugu.

Phase 4 - Safety-sensitive operations

True redaction, sanitization, suspicious-content preflight, compare foundation and structured extraction.

Phase 5 - Archival and digital signatures

PDF/A pipeline/validation, certificate signing/verification, print center.

Phase 6 - Automation

Saved workflows, CLI, watched folders and .dsflow exchange.

Phase 7 - Optional document intelligence

Local model manager, summary, Ask Document, classification and structured extraction with page citations.

Phase 8 - Product hardening

Accessibility audit, localization, signed updates, clean-machine packaging, benchmark regression gates.

Phase 9+ - Web/mobile/plugin ecosystem

Optional cloud control plane, browser-local router, mobile capture companion, signature requests and plugin SDK.

Figure 8. Milestone roadmap.

20. Codex Delivery Method

Vertical-slice template

Each Codex task should include:

- User-visible behavior.
- Manifest/schema changes.
- Rust command/adapter.
- Job progress, cancel, verification and cleanup.
- UI state and error copy.
- Unit/integration/failure tests.
- Documentation and ADR.
- Benchmark evidence if the hot path changed.

Prompt discipline

- Give Codex one milestone or one feature slice at a time.
- Require it to inspect the repository and propose a file-level plan before editing.
- Require exact commands and test output.
- Require screenshots for UI changes.
- Stop the task when acceptance criteria are met; do not let it expand scope.

First real feature

Implement PDF merge after Phase 0. It exercises multiple inputs, inspection, page counts, ordering, qpdf execution, cancellation, output naming, verification, cleanup and history without complex editing coordinates.

21. Figma Handoff

File pages

1. Cover & decisions
2. Foundations
3. Variables and tokens
4. Components
5. Patterns
6. Desktop flows
7. Web adaptation
8. Mobile capture
9. Prototypes
10. Accessibility and redlines
11. Archive

Variable collections

- Color / Light
- Color / Dark
- Color / High Contrast
- Spacing
- Radius
- Typography
- Elevation
- Motion
- Density

Core components

App shell, navigation item, command search, drop zone, tool card, file row, page thumbnail, canvas toolbar, inspector section, form controls, segmented control, locality badge, status chip, progress row, job tray, result summary, empty/error state, toast and dialog.

Plugins/workflow

- Tokens Studio: import/sync token JSON.
- Iconify: explore/import icons; verify each icon-set license and normalize to a small chosen set.
- Design Lint: find missing styles/inconsistent values.
- Accessibility checker: contrast and naming checks.
- Autoflow/equivalent: document user-flow arrows only, not production components.

Figma-to-code rule

Figma is the design source for behavior/layout and components; tokens JSON is the shared source for numeric values. Do not copy absolute pixel positions from screenshots into production code.

22. Notion Knowledge Base Structure

Recommended page tree

Use one private parent page named Document Studio - Project Hub. Create these child pages under it:

- 00 Audit and decisions
- 01 Product charter
- 02 Feature catalogue
- 03 Personas and workflows
- 04 UI/UX and Figma handoff
- 05 System architecture
- 06 Data, API and operation contract
- 07 Security and privacy
- 08 Performance plan
- 09 Dependencies and model registry
- 10 Roadmap
- 11 Codex prompts and implementation log
- 12 Testing and release checklist
- 13 Research archive
- 14 Diagrams and whiteboards
- 15 Development setup
- 16 Final recheck and publication boundary

Databases

- Features: category, phase, priority, status, owner, acceptance evidence.
- Decisions/ADRs: status, date, decision, alternatives, consequences.
- Milestones/tasks: phase, status, dependency, acceptance criteria.
- Dependencies/models: version/revision, license, status, benchmark, risk.
- Test cases: feature, platform, fixture, expected result, evidence.

The notion-import folder contains Markdown, CSV and attachments that mirror this structure. It includes the current master blueprint, the full research input, the historical reports, reference screens and all whiteboard diagrams.

23. Implementation Readiness Checklist

Identity and scope

- [DONE] Canonical name is Document Studio.
- [DONE] Desktop-first platform strategy chosen.
- [DONE] Unified feature catalogue created.
- [DONE] Personal edition has no artificial plan split.

Architecture

- [DONE] Tauri/Rust desktop boundary chosen.
- [DONE] Operation lifecycle and job state model defined.
- [DONE] Local SQLite and future cloud storage separated.
- [DONE] Browser/local/cloud routing documented.
- [DONE] Security/privacy and recovery rules documented.

Design

- [DONE] Information architecture defined.
- [DONE] Screen specifications and core components defined.
- [DONE] Tokens and Figma handoff included.
- [DONE] Accessibility criteria included.
- [DONE] Prototype included.

Engineering

- [DONE] Repository scaffold and AGENTS file included.
- [DONE] Codex prompts included.
- [DONE] Dependency/model registers included.
- [DONE] Validation scripts and CI skeleton included.
- [TODO] Create private GitHub repository and push this package.
- [TODO] Create/import live Notion knowledge base.
- [TODO] Create live Figma file from the handoff.
- [TODO] Run toolchain installation on the target Windows machine.
- [TODO] Execute Phase 0 prompt in Codex.

Proceed/no-proceed

Proceed to Phase 0 only after the five unchecked environment/publication items are confirmed. The planning package itself is complete enough to start.

Final local recheck

The local package validation and publication boundary are recorded in ../FINAL_RECHECK.md. The unchecked items above require connected external accounts or the target development machine; they are not silently marked complete.

24. Development Setup and First Build

This is the target-machine runbook for the Windows-first Phase 0 build. It intentionally separates development prerequisites, core engine packages, and later optional capabilities so Codex does not install a large, fragile toolchain before it is needed.

Target baseline

- Windows 11 x64, current security updates.
- 16 GB RAM minimum; 32 GB recommended for OCR, very large documents and local models.
- SSD with at least 20 GB free for toolchains, build caches, fixtures and temporary jobs.
- A private GitHub repository and a non-administrator day-to-day development account.

1. Install Tauri development prerequisites

Tauri requires Microsoft C++ Build Tools and Microsoft Edge WebView2 on Windows. In Visual Studio Build Tools, enable Desktop development with C++. WebView2 is normally already present on current Windows 10/11 systems. MSI packaging may also require the Windows VBSCRIPT optional feature.

Install the remaining tools from an elevated PowerShell window:

```
winget install --id Git.Git -e
winget install --id Rustlang.Rustup -e
winget install --id OpenJS.NodeJS.LTS -e
winget install --id Python.Python.3.12 -e
rustup default stable-msvc
```

Restart the terminal, then verify:

```
git --version
node --version
npm --version
rustc --version
cargo --version
python --version
```

2. Clone and validate the planning repository

```
git clone <PRIVATE_REPOSITORY_URL> document-studio
cd document-studio
python scripts/validate_repo.py
python scripts/check_links.py
npm install
npm run typecheck
cargo test --manifest-path apps/desktop/src-tauri/Cargo.toml
npm --workspace apps/desktop run tauri dev
```

The first successful checkpoint is the starter window plus the system_status Tauri command. Do not add PDF engines until this baseline builds and tests cleanly.

3. Core engine installation policy

qpdf

Use the signed official Windows release. Verify its published checksum/signature, store provenance in third_party/manifest.json, and invoke it through validated argument arrays. It is the Phase 1 engine for structural transforms, encryption, linearization and integrity checks.

libvips

Use a reviewed Windows binary or a reproducible build. Prefer dynamic linking and ship the required LGPL notices/source-offer information. Use it for image conversion, thumbnails and image-heavy compression paths.

PDF.js

Install through npm. Keep rendering in the UI process, but move expensive page inspection and document operations to workers/Rust commands. Use virtualized thumbnails and progressive page rendering.

4. Optional capability dependencies

LibreOffice

Treat LibreOffice as a separately installed dependency during development. Detect its version/path, create an isolated user profile per worker, impose time/memory limits and verify every converted output. Decide later whether the installer downloads it, bundles a permitted subset, or requires the user to install it.

OCRmyPDF and Tesseract

Do not require OCR in Phase 0. For the OCR milestone:

1. Benchmark direct Tesseract + controlled PDF assembly against OCRmyPDF.
2. For OCRmyPDF on native Windows, provision 64-bit Python, Tesseract and Ghostscript in a managed worker environment.
3. Treat Windows third-party Tesseract binaries as a provenance risk; record source, checksum and update channel.
4. Install only the requested language packs (eng, hin, tel) and verify checksums.
5. Keep WSL/Docker as development or optional-server paths, not silent desktop requirements.

Local AI models

No model is installed during the foundation. The model manager must require user action, pin an immutable revision, verify checksums, show disk/memory requirements and allow removal. Run the benchmarks in models/models.yaml before enabling any model-backed feature.

5. Phase 0 command sequence for Codex

```
1. Open the repository root in Codex.
2. Read AGENTS.md, CODEX_START_HERE.md and FINAL_RECHECK.md.
3. Run codex/prompts/01-foundation.md only.
4. Make one vertical slice at a time.
5. Run format, typecheck, Rust tests, repository validators and a Tauri smoke test.
6. Record command output and unresolved decisions under docs/implementation-log/.
7. Stop when the Phase 0 acceptance gate passes.
```

6. First-build evidence to retain

- Exact Node/npm/Rust/Cargo versions.
- Generated lockfiles and Cargo.lock.
- npm run typecheck output.
- cargo test output.
- Tauri dev-launch screenshot.
- Dependency diagnostic screen output.
- Phase 0 acceptance checklist and any deviations.

Never mark the toolchain ready based on installation alone; preserve reproducible evidence that the starter builds and runs on the target machine.

25. Document Studio - Final Recheck Record

Recheck date: 22 July 2026 Canonical product name: Document Studio Package version: 2.0.1-final-recheck

Final verdict

The earlier deliverables were not fully ready because the historical report still used the former working name and live publication to Notion, Canvs, Figma, GitHub and Hugging Face had not been verified. This recheck closes the package-level gaps and distinguishes completed local deliverables from external actions that require connected accounts or the target development machine.

Completed and rechecked

- Canonical naming is Document Studio in all current product, architecture, UI/UX and Codex documents.
- A single unified catalogue contains 132 capabilities without free-versus-paid product tiers.
- Product charter, personas, platform strategy, information architecture, screen specifications, performance budgets, system architecture, operation contract, database design, API/IPC contracts, threat model, dependencies, model policy, testing, CI/CD, roadmap and Codex delivery method are documented.
- The desktop starter includes Tauri 2, React, TypeScript, Vite 8 and a Rust boundary scaffold.
- The Vite React plugin was aligned with Vite 8 (@vitejs/plugin-react 6.x), and the desktop build script now performs a no-emit TypeScript check before bundling.
- A browser-openable interactive prototype, design tokens, Figma component inventory, Figma plugin plan, reference screens and prototype flows are included.
- Architecture, information architecture, job lifecycle, execution-routing and roadmap whiteboards are available as source plus PNG/SVG exports.
- Hugging Face candidates are recorded with explicit evaluation gates; no model is silently downloaded or treated as production-ready.
- GitHub-ready repository structure, CI workflow, ADRs, validation scripts, AGENTS instructions and staged Codex prompts are included.
- The Notion import package contains current documentation, the master DOCX/PDF, full feature CSV, source research, historical reports, prototype screenshots and all exported whiteboards.
- The printable master report was rendered and visually inspected page by page before packaging.

Validation completed in this environment

- Repository structure and feature-count validator.
- Internal Markdown-link checker.
- JSON and YAML parsing.
- JavaScript syntax check for the static prototype.
- Go control-plane skeleton unit test.
- DOCX ZIP integrity and accessibility audit.
- PDF openability, page count and text extraction.
- Canonical-name scan across current sources, with historical files excluded by design.
- The frontend package manifest and TypeScript/Vite configuration were statically reviewed. Full npm installation/typecheck/build did not run because registry resolution exceeded the environment timeout.
- Rust/Cargo compilation did not run because the Rust toolchain is not installed in this environment.

External or target-machine actions still required

These are execution/publication steps, not missing planning work:

1. Create a private GitHub repository and push the package.
2. Import/publish the prepared knowledge base in the connected Notion workspace.
3. Create the live Figma file from the supplied design tokens and handoff.
4. Import or recreate the supplied diagrams in the connected Canvs board.
5. Create the Hugging Face collection/Space only after model evaluations pass.
6. Install Rust/Cargo, qpdf, OCR, Office-conversion and packaging dependencies on the target Windows machine.

7. Run the Phase 0 Codex prompt and capture build/test evidence.

Proceed gate

The planning, design and starter package is complete. Production implementation has not been claimed. Begin Phase 0 only after the repository and target toolchain are available; publish the prepared external-app packages when those connectors are connected.

Appendix A. Full unified feature catalogue

The following 132 capabilities are planned as one complete product. Phase expresses sequence, not a subscription tier. “Must/Should/Could” is delivery priority.

Category	Feature	Required behavior	Phase / priority / engine
Organize	Merge PDF	Combine PDFs/images, page-range imports, drag reorder, bookmark and outline preservation	P1 · Must · qpdf
Organize	Split by ranges	Create files from explicit page ranges	P1 · Must · qpdf
Organize	Split every N pages	Split at a fixed page interval	P1 · Must · qpdf
Organize	Split by bookmarks	Use document outline levels as boundaries	P2 · Should · qpdf
Organize	Split by target size	Estimate boundaries to produce size-bounded parts	P3 · Could · qpdf + estimator
Organize	Extract pages	Export selected pages as one PDF, individual PDFs, or ZIP	P1 · Must · qpdf
Organize	Remove pages	Delete thumbnail/range selections with undo before export	P1 · Must · qpdf
Organize	Reorder pages	Drag, reverse, odd/even regroup, duplicate, and move pages	P1 · Must · qpdf
Organize	Rotate pages	Rotate selected/odd/even/all pages 90/180/270 degrees	P1 · Must · qpdf
Organize	Insert blank pages	Insert configurable blank page sizes and orientation	P2 · Should · pdf-lib/qpdf
Organize	Interleave documents	Alternate pages from two or more files	P2 · Should · qpdf
Organize	N-up and booklet	Impose pages for 2-up, 4-up, and booklet printing	P5 · Could · pdfcpu/print engine
Organize	Scan to PDF	Camera/scanner import, crop, dewarp, cleanup, reorder	P6 · Should · OpenCV + native camera
Optimize	Compress - lossless	Remove redundant structures and optimize streams without image loss	P1 · Must · qpdf
Optimize	Compress - balanced	Downsample/re-encode images with readable defaults	P2 · Must · libvips + qpdf
Optimize	Compress - strong	Aggressive image and font optimization with preview comparison	P2 · Should · libvips + qpdf
Optimize	Image-aware compression	Per-image DPI, JPEG quality, monochrome and transparency controls	P2 · Should · libvips
Optimize	Linearize PDF	Fast-web-view optimization	P1 · Should · qpdf
Optimize	Repair PDF	Rebuild xref/object streams and salvage readable pages	P2 · Must · qpdf + fallback parser
Optimize	Remove unused objects	Garbage collect orphaned objects/resources	P2 · Should · qpdf
Optimize	PDF/A conversion	Create archival variants and validate conformance	P5 · Should · LibreOffice/veraPDF
Optimize	Optimize scans	Deskew, despeckle, background cleanup, bilevel optimization	P3 · Should · OCRmyPDF + unpaper

Category	Feature	Required behavior	Phase / priority / engine
Convert to PDF	JPG/PNG/WebP to PDF	Convert raster images with page size, margins, orientation, quality	P1 · Must · libvips + PDF writer
Convert to PDF	TIFF/BMP/HEIC to PDF	Convert additional image formats and multipage TIFF	P2 · Should · libvips/ImageMagick
Convert to PDF	Word to PDF	DOC/DOCX/ODT conversion	P2 · Must · LibreOffice
Convert to PDF	Excel to PDF	XLS/XLSX/ODS conversion with sheet/range options	P2 · Must · LibreOffice
Convert to PDF	PowerPoint to PDF	PPT/PPTX/ODP conversion	P2 · Must · LibreOffice
Convert to PDF	Markdown to PDF	Render Markdown with print themes and syntax highlighting	P2 · Should · Chromium/HTML pipeline
Convert to PDF	HTML to PDF	Render local HTML or approved URLs with page controls	P2 · Should · Chromium/Gotenberg
Convert to PDF	Text to PDF	Convert TXT/CSV/JSON/XML to formatted PDF	P2 · Should · native renderer
Convert to PDF	Email to PDF	Convert EML/MSG with headers and attachments index	P5 · Could · Docling/mail parser
Convert to PDF	SVG to PDF	Preserve vector content when possible	P3 · Should · resvg/Cairo
Convert to PDF	Archive package to PDF	Combine mixed supported inputs into an ordered PDF binder	P4 · Could · workflow engine
Convert from PDF	PDF to JPG/PNG/WebP	Render selected pages at configurable DPI	P1 · Must · PDFium/MuPDF/libvips
Convert from PDF	PDF to TIFF	Export single/multipage TIFF	P2 · Should · PDFium + libvips
Convert from PDF	PDF to text	Reading-order text extraction with OCR fallback	P1 · Must · PDF.js/Docling/Tesseract
Convert from PDF	PDF to Markdown	Structured headings, lists, tables, images and links	P4 · Should · Docling/Granite Docling
Convert from PDF	PDF to Word	Layout-aware DOCX with OCR fallback and confidence warnings	P5 · Should · Docling + DOCX writer
Convert from PDF	PDF to Excel	Detect tables; export CSV/XLSX with review step	P5 · Should · Docling/table engine
Convert from PDF	PDF to PowerPoint	Page-as-slide and editable reconstruction modes	P6 · Could · renderer + PPTX writer
Convert from PDF	PDF to HTML	Structured HTML with page/image assets	P4 · Could · Docling
Convert from PDF	Extract images	Export embedded images without unnecessary re-rendering	P2 · Should · qpdf/PDF parser
Convert from PDF	Extract attachments	List and export embedded files safely	P2 · Should · qpdf/PDF parser
Convert from PDF	Extract fonts report	Inventory embedded fonts without redistributing font binaries	P5 · Could · PDF parser
Convert from PDF	Extract annotations	Export comments/highlights to JSON/CSV/FDF-like formats	P4 · Could · PDF parser
Convert from PDF	PDF to searchable archive	OCR + PDF/A + metadata workflow	P5 · Should · workflow engine
Edit	Add text	Positioned text overlays with font embedding and	P3 · Must · pdf-lib/PDF editor

Category	Feature	Required behavior	Phase / priority / engine
		alignment	
Edit	Edit overlay text	Move/resize/delete objects created by Document Studio	P3 · Must · editor scene graph
Edit	Add shapes	Rectangles, ellipses, lines, arrows, polygons and callouts	P3 · Must · editor scene graph
Edit	Freehand draw	Pen/highlighter with smoothing, opacity and eraser	P3 · Should · canvas + PDF annotations
Edit	Highlight/underline/strikeout	Text-aware markup annotations	P3 · Must · PDF.js text layer
Edit	Comments and notes	Sticky notes and threaded local comments	P3 · Should · PDF annotations
Edit	Insert images	Place, crop, rotate and resize images/logos/stamps	P3 · Must · pdf-lib/PDF editor
Edit	Add page numbers	Position, format, range, start number and exclusions	P2 · Must · PDF overlay engine
Edit	Headers and footers	Templates using date, filename, page and page count	P3 · Should · PDF overlay engine
Edit	Watermark	Text/image watermark with opacity, angle and page scope	P2 · Must · PDF overlay engine
Edit	Crop pages	Visual crop boxes, margins and uniform/apply-to-range modes	P3 · Must · PDF page boxes
Edit	Resize pages	Change page media size and optionally scale content	P4 · Should · PDF page transform
Edit	Redact	Search/manual true redaction that removes underlying content	P4 · Must · MuPDF/PDF redaction engine
Edit	Sanitize after redaction	Remove metadata, hidden objects, attachments and scripts	P4 · Must · qpdf + sanitizer
Edit	Compare PDFs	Side-by-side, overlay and semantic text difference report	P5 · Should · renderer + diff engine
Edit	Replace pages	Swap selected pages while preserving order/bookmarks when possible	P3 · Should · qpdf
Edit	Layers/optional content	Inspect and toggle optional-content groups	P7 · Could · advanced PDF parser
Edit	Bates numbering	Legal document numbering with prefix/suffix and range rules	P5 · Could · overlay engine
Forms	Fill AcroForms	Text, checkbox, radio, choice, date and signature fields	P3 · Must · PDF form engine
Forms	Create form fields	Add form controls, names, validation, required state and tab order	P5 · Should · PDF form engine
Forms	Detect form fields	Suggest fields on flattened/scanned forms	P7 · Could · layout model
Forms	Flatten forms	Bake appearances into final pages	P3 · Must · PDF form engine
Forms	Import/export form data	FDF/XFDF/JSON-like data interchange	P6 · Could · form data adapter
Forms	Form validation report	Missing required values and field constraints	P5 · Should · form engine
Forms	Template forms	Save reusable field layouts for recurring documents	P6 · Could · workflow + form engine

Category	Feature	Required behavior	Phase / priority / engine
OCR	Searchable PDF	Add hidden text layer while preserving page appearance	P3 · Must · OCRmyPDF + Tesseract
OCR	English OCR	Default English language pack	P3 · Must · Tesseract eng
OCR	Hindi OCR	Hindi language pack	P3 · Must · Tesseract hin
OCR	Telugu OCR	Telugu language pack with benchmark against PaddleOCR	P3 · Must · Tesseract tel / benchmark
OCR	Mixed-language OCR	Run multiple languages in one job	P3 · Must · Tesseract
OCR	Deskew/orientation	Detect page rotation and deskew	P3 · Must · OCRmyPDF
OCR	Cleanup scans	Denoise, remove background and optimize before OCR	P3 · Should · OCRmyPDF/unpaper
OCR	OCR confidence review	Page-level warnings and low-confidence text overlay review	P6 · Could · OCR pipeline
OCR	hOCR/ALTO export	Export OCR geometry and confidence	P6 · Could · Tesseract
OCR	Handwriting research mode	Optional experimental handwriting model; never promise accuracy	P8 · Could · HF model evaluation
Security	Protect PDF	Set open password, owner password and permissions	P2 · Must · qpdf
Security	Unlock PDF	Remove protection only with valid user-supplied credentials	P2 · Must · qpdf
Security	AES encryption	Support modern PDF encryption profiles	P2 · Must · qpdf
Security	Metadata editor	View, change and remove title/author/subject/keywords/dates	P2 · Must · qpdf/PDF parser
Security	Sanitize document	Remove scripts, actions, embedded files, selected annotations and hidden metadata	P4 · Must · sanitizer
Security	Malware-safe preflight	Detect suspicious actions/attachments and isolate risky files	P4 · Should · static scanner
Security	Secure erase temporary jobs	Best-effort cleanup and encrypted temp option	P4 · Should · OS storage layer
Security	Certificate validation	Inspect signatures, certificate chain and revocation where available	P5 · Should · PAdES verifier
Security	Permission report	Explain permissions and encryption status in plain language	P2 · Must · qpdf
Security	Privacy mode	Globally disable network and external providers	P0 · Must · platform policy
Sign	Draw/type/upload signature	Create reusable local signature assets	P3 · Must · editor + OS keychain
Sign	Place signature/initial/date	Drag placement and repeat across pages	P3 · Must · PDF overlay engine
Sign	Signature vault	Encrypt local identities and assets using OS credentials	P3 · Must · OS keychain
Sign	Certificate signing	PAdES/CMS-based digital signatures	P5 · Should · standards-compliant signer

Category	Feature	Required behavior	Phase / priority / engine
Sign	Timestamp signing	Optional trusted timestamp integration	P6 · Could · TSA adapter
Sign	Signature validation	Show signer, integrity, trust and timestamp status	P5 · Should · PAdES verifier
Sign	Request signatures	Cloud-only multi-recipient workflow, optional future module	P9 · Could · cloud e-sign service
Automation	Batch processing	Apply one operation to many files with isolated failures	P2 · Must · job engine
Automation	Saved workflows	Chain typed operations with variables and review gates	P6 · Should · workflow engine
Automation	Watched folders	Run approved workflows for new files	P6 · Should · desktop watcher
Automation	Command-line interface	Expose stable operations for scripts	P6 · Should · Rust CLI
Automation	Local HTTP API	Optional loopback API with token protection	P7 · Could · Rust local API
Automation	Naming templates	Deterministic outputs with collision policies	P1 · Must · job engine
Automation	Presets	Reusable per-tool settings	P1 · Must · SQLite
Automation	Schedule jobs	Run local workflows at a chosen time	P7 · Could · OS scheduler
Automation	Share workflow file	Portable .dsflow JSON with versioned schema	P6 · Should · workflow engine
Automation	Hot folders with routing	Route by filename, type or detected document class	P8 · Could · workflow + classifier
AI	Document summary	Local-first summary with selectable pages and citations	P7 · Should · provider-neutral LLM
AI	Ask document	RAG over extracted text with page citations	P7 · Should · multilingual-e5 + LLM
AI	Classify and rename	Suggest type, filename and destination	P7 · Should · classifier/LLM
AI	Structured extraction	Invoices, receipts, forms and custom JSON schema	P7 · Should · Docling + LLM
AI	Table understanding	Detect/repair table structure for export	P7 · Should · Docling/Granite Docling
AI	Translate document	Translate text with clear layout-fidelity limitations	P8 · Could · translation provider
AI	Smart split	Suggest logical document boundaries	P8 · Could · layout model + rules
AI	Sensitive-data finder	Suggest PII/financial identifiers for manual review	P7 · Should · NER/rules
AI	AI-assisted redaction review	Never auto-commit; user confirms every redaction	P8 · Could · NER + review UI
AI	Document Q&A export	Export questions/answers with source-page references	P8 · Could · RAG engine
AI	Model download manager	Install, verify, update and remove local models	P7 · Should · model registry
AI	External-provider consent	Per-job approval with provider, scope and retention notice	P7 · Must · policy layer
Platform	Recent work and history	Search jobs, inspect settings, rerun, reveal output and clean history	P0 · Must · SQLite
Platform	Command palette	Keyboard-first access to tools, files and presets	P0 · Must · React UI

Category	Feature	Required behavior	Phase / priority / engine
Platform	Dependency diagnostics	Detect missing engines, versions and remediation steps	P0 · Must · Rust core
Platform	Crash recovery	Recover interrupted jobs without false success	P0 · Must · job engine
Platform	Update manager	Signed releases with optional staged rollout	P8 · Should · Tauri updater
Platform	Localization framework	English first; Telugu and Hindi UI-ready strings	P0 · Should · i18n
Platform	Accessibility mode	Keyboard, screen reader, 200% scaling, high contrast, reduced motion	P0 · Must · design system
Platform	Theme modes	Light, dark and high-contrast variables	P0 · Should · design tokens
Platform	Print center	Preview, grayscale, duplex guidance, N-up, booklet and fit	P5 · Should · print pipeline
Platform	Cross-device handoff	Optional encrypted handoff between personal devices	P10 · Could · future cloud sync
Platform	Plugin SDK	Versioned operation manifests and sandboxed adapters	P9 · Could · operation SDK
Platform	Diagnostics export	Logs and dependency data without document content/secrets	P0 · Must · platform

Appendix B. Codex prompt pack

Codex Prompt 01 - Foundation

Read AGENTS.md, CODEX_START_HERE.md, docs/09-SYSTEM-ARCHITECTURE.md, docs/10-OPERATION-CONTRACT.md and docs/22-IMPLEMENTATION-READINESS-CHECKLIST.md before editing.

Implement Phase 0 only:

- Make the Tauri/React shell run.
- Implement design-token consumption and Home layout parity with the prototype.
- Add SQLite migrations for jobs, inputs, outputs, presets, workflows, dependencies and settings.
- Implement the typed job state machine.
- Implement secure per-job workspaces and startup reconciliation.
- Implement diagnostic.copy through inspect, preflight, estimate, execute, cancel, verify, publish, audit and cleanup.
- Add dependency diagnostics.
- Add tests for completion, cancel, failure, crash reconciliation and path safety.

Before editing, give a file-by-file plan. After editing, run formatting, typecheck, Rust/unit/integration tests and a development smoke test. Stop when Phase 0 acceptance criteria pass.

Codex Prompt 02 - PDF Merge

Implement pdf.merge as the first production operation using qpdf. Support multiple inputs, page-range selection, drag order, outline/bookmark policy, deterministic naming, cancellation, staged output, verification and history. Do not overwrite originals. Add corrupt/encrypted input errors and golden/cancel/collision/recovery tests.

Codex Prompt 03 - Core PDF operations

After merge is accepted, add split, extract, remove, reorder, rotate and linearize through the same operation contract. Reuse page-range parsing, file inspection, qpdf execution, verification and UI patterns. Add fixtures and benchmarks.

Codex Prompt 04 - Compression and conversion

Add lossless compression first, then image-aware balanced compression using libvips. Add images-to-PDF, PDF-to-images and Office-to-PDF through separately diagnosed dependencies. Include before/after size and representative visual comparison.

Codex Prompt 05 - OCR and security

Add OCRmyPDF/Tesseract with English, Hindi and Telugu packs, followed by protect/unlock/metadata. Use representative-page preview, stage progress, cancellation and operation-specific verification. External services remain disabled.

Codex Prompt 06 - Editor and true redaction

Implement overlays/annotations/crop before true redaction. Keep an editor scene graph separate from committed PDF content. Redaction must remove recoverable content and pass extraction plus visual verification; a black overlay is unacceptable.

Codex Prompt 07 - Automation and optional AI

Implement presets, saved workflows, CLI and watched folders. Only after those are stable, implement the model manager and evaluate Docling/Granite Docling and multilingual-e5-small. AI answers require source pages and external providers require consent.

Appendix C. Primary implementation references

- Tauri project and security/capability documentation: https://tauri.app/
- React 19.2 release: https://react.dev/blog/2025/10/01/react-19-2
- Vite 8.1 announcement: https://vite.dev/blog/announcing-vite8-1
- qpdf: https://github.com/qpdf/qpdf
- PDF.js: https://github.com/mozilla/pdf.js
- pdf-lib: https://github.com/Hopding/pdf-lib
- pdfcpu: https://github.com/pdfcpu/pdfcpu
- libvips: https://github.com/libvips/libvips
- OCRmyPDF: https://github.com/ocrmypdf/OCRmyPDF
- Tesseract: https://github.com/tesseract-ocr/tesseract
- LibreOffice: https://www.libreoffice.org/
- Gotenberg: https://github.com/gotenberg/gotenberg
- Docling: https://github.com/docling-project/docling
- Granite Docling 258M: https://huggingface.co/ibm-granite/granite-docling-258M
- multilingual-e5-small: https://huggingface.co/intfloat/multilingual-e5-small
- UI UX Pro Max: https://github.com/nextlevelbuilder/ui-ux-pro-max-skill
- Tokens Studio: https://github.com/tokens-studio/figma-plugin
- Design Lint: https://github.com/destefanis/design-lint
- Iconify Figma: https://github.com/iconify/iconify-figma

Companion package contents

- Codex-ready repository scaffold
- Interactive static UI prototype and two reference screens
- Figma tokens, file structure, components and plugin plan
- Hugging Face/local-model registry
- Architecture and workflow diagrams in DOT, Mermaid, PNG and SVG
- Notion import pack with source research and historical reports
- Validation scripts, CI skeleton, ADRs and acceptance checklists